ASSESSMENT 1

Movie Performance Management System

Individual Component Design & Governance Plan

REST API Module of the Movie Performance Management System

Name	Chandani Gupta
Team role	Data and API Platform Team: REST API and openapi.yaml
Assigned module	API application (controllers, services, validation, error handling)
Team members	Prakash, Rose, Chandani, Reinaldo, Avishek (Avi)
Date	October 2026

Table of Contents

1. Introduction **1**
1.1 Purpose and Scope 1
1.2 Source Documents 1
2. Component Design **2**
2.1 Position within the Architecture 2
2.2 Internal Structure 2
2.3 Design Principles 3
2.4 Standard Error Handling 3
3. API Endpoint Responsibilities **5**
3.1 Endpoint Summary 5
3.2 Movie Endpoints and Other Additions to the Contract 6
3.3 Open Decisions and Dependencies 6
4. Risk Management and Delivery Roadmap **7**
4.1 Risk Register 7
4.2 Risk Assessment 7
4.3 Delivery Roadmap 8
4.4 Verification and Testing Plan 9
5. Governance and Alignment with Team Git Rules **10**
5.1 Position in the Team Structure 10
5.2 Git Working Rules 10
5.3 Contract Change Process 10
5.4 Communication and Review 11
6. Conclusion **12**
References **13**

List of Tables

Table 1: Interfaces of the API application....2
Table 2: Components of the API application and their responsibilities....3
Table 3: HTTP status codes returned by the API....4
Table 4: Endpoints owned by the API module....5
Table 5: Additions and choices in the final contract beyond the Blueprint....6
Table 6: Personal risk register for the API module....7
Table 7: Delivery phases, deliverables and exit checks....9
Table 8: Git working rules applied to the API module....10

List of Figures

Figure 1: Component structure of the API application....2
Figure 2: Risk matrix for the API module....8
Figure 3: Phased delivery roadmap....8
Figure 4: Contract change process....11

1. Introduction

1.1 Purpose and Scope

This plan defines how I will design, build and govern my assigned module of the Movie Performance Management System: the REST API application. It is drawn from the team's Architecture Blueprint and from the OpenAPI 3.0 contract (openapi.yaml) held in the main folder of our shared Git repository.

The API is the sole route between the web application and the relational database. All rules about data are therefore enforced in one place, and the pages built by Prakash and Avishek depend directly on its behaviour. The final contract, openapi.yaml version 1.0.0, defines 13 operations on 11 paths under /api/v1. The plan covers four areas: the design of the module (Section 2), the endpoints for which I am responsible (Section 3), a personal risk and delivery roadmap (Section 4), and my alignment with the team's Git and contract-change rules (Section 5).

1.2 Source Documents

The plan relies on the following parts of the Architecture Blueprint: the glossary (Blueprint Tables 1 and 2), the event storming board and domain events (Blueprint Table 3), the C4 container and component diagrams (Blueprint Figures 3 to 5), the endpoint groups (Blueprint Table 6), the team structure (Blueprint Section 4) and the shared rules for errors, authentication and conventions (Blueprint Section 5). Tables and figures in this plan are numbered separately, and any reference to the Blueprint is labelled as such.

2. Component Design

2.1 Position within the Architecture

In the C4 container view, the system has three containers: the web application, the API application and the relational database. My module is the middle container. It receives REST requests over HTTPS with JSON bodies, validates them against the contract, applies the review rule and returns JSON responses. It reaches the database only through a data access layer built on the schema and queries provided by Rose.

Table 1: Interfaces of the API application

Neighbour	Direction	Interface
Web application (Prakash, Avishek)	Calls the API	HTTPS, REST and JSON (UTF-8), with shapes defined in openapi.yaml
Relational database (Rose)	Called by the data access layer	SQL through a database driver
Git repository (Reinaldo as administrator)	Reviews and merges changes	Pull requests to the main branch

2.2 Internal Structure

Figure 1 shows the internal structure of the API application, following the C4 component view in the Blueprint. A single validation layer sits in front of the controllers. Each group of endpoints has one controller, and services hold the business rules where logic is required. Only the data access layer communicates with the database. The authentication components are dashed because login is optional.

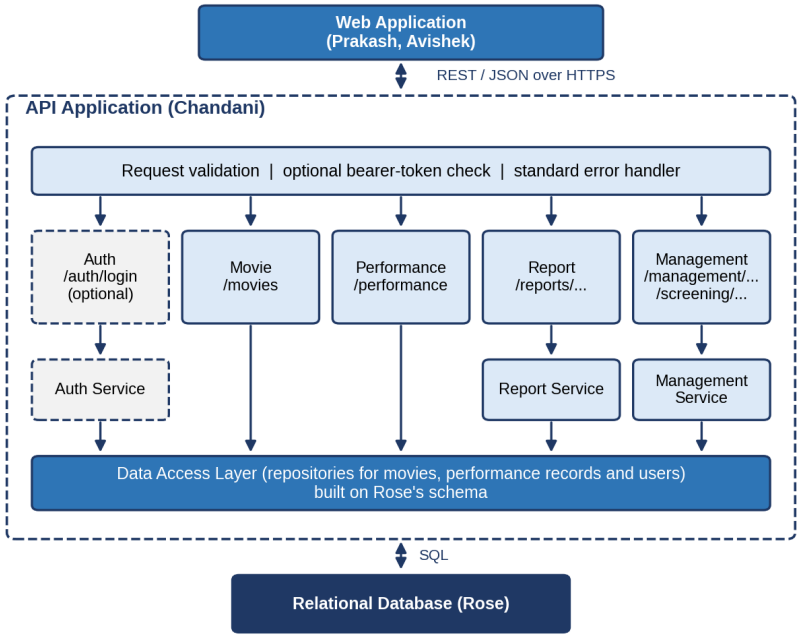


Figure 1: Component structure of the API application

Table 2 describes the responsibility of each component. The movie and performance controllers call the data access layer directly because they only create, read, update and delete records. The report and management services hold the weekly summary logic and the review rule respectively.

Table 2: Components of the API application and their responsibilities

Component	Responsibility	Design note
Validation, token check and error handler	Checks formats, required fields and values; verifies the token when login is enabled; converts every failure into the standard Error object	Single point for all error handling
Movie controller (/movies)	Lists movies, creates a movie and returns one movie by its ID	Calls data layer directly
Performance controller (/performance)	Creates, updates and deletes weekly performance records	Enforces one record per movie per week
Report controller and service (/reports)	Builds the weekly report and the revenue-trend chart data	Service holds summary logic
Management controller and service	Calculates review flags and records extend or end decisions	Service holds the review rule
Auth controller and service (optional)	Checks credentials and issues a bearer token	Built only if login is approved
Data access layer	Repositories for movies, performance records and users	Only component that touches the database

2.3 Design Principles

Five principles guide the implementation.

- **API-first.** The contract is agreed before the code is written, so the pages can be built against the examples in openapi.yaml while the API is still in progress.
- **One place for rules.** Validation, error format and the review rule live in the API only; the web application does not duplicate them.
- **Derived flags.** Movie Flagged for Review is a derived event, calculated when a manager requests flags for a given week rather than stored. This keeps the data model small and avoids stale flags.
- **Shared conventions.** All endpoints sit under /api/v1. Bodies are JSON in UTF-8. Dates use ISO 8601 (YYYY-MM-DD) and a week is identified by its Monday (weekStart). Money is a non-negative AUD amount with up to two decimal places. Field names use camelCase and identifiers are whole numbers.
- **Safe errors.** Messages never reveal database queries or stack traces; these are written to the server log (OWASP Foundation, 2023).

2.4 Standard Error Handling

Every failure returns the Error object defined in openapi.yaml, with a machine-readable code and a human-readable message. HTTP status codes keep their standard meanings (Fielding et al., 2022). Table 3 lists the codes the API will return and the situations in which it will return them.

Table 3: HTTP status codes returned by the API

Status	Error code	Returned when
200 OK	n/a	A read, update or action succeeds
201 Created	n/a	A resource is created (POST /performance, POST /movies)
204 No Content	n/a	A DELETE succeeds
400 Bad Request	BAD_REQUEST	A date has the wrong format, revenue is negative or weekStart is missing
401 Unauthorized	UNAUTHORIZED	The token is missing or invalid, or login fails (only when login is enabled)
404 Not Found	NOT_FOUND	A movieId or recordId does not exist
409 Conflict	CONFLICT	A second record is submitted for the same movie and week, or a screening that has ended is extended
500 Internal Server Error	SERVER_ERROR	An unexpected failure occurs on the server

3. API Endpoint Responsibilities

3.1 Endpoint Summary

The Blueprint assigns me the implementation of every endpoint group, with Rose providing the data layer for all groups except authentication. I have also added a Movies group, explained in Section 3.2. Table 4 expands each operation with its purpose, the main rules I must enforce and the teammate who consumes it.

Table 4: Endpoints owned by the API module

Endpoint	Purpose	Key rules and error responses	Used by
Movies			
GET /movies	List movies	200 with a list of movies (movieId, title, genre, releaseDate, screeningStatus)	Prakash
POST /movies	Add a movie	Requires title, genre and releaseDate. 201 on success; 400 for invalid input	Prakash
GET /movies/{movieId}	Get one movie	200 on success; 404 for unknown movie	Prakash
Performance			
POST /performance	Record weekly revenue (Weekly Revenue Recorded)	Validates movieId, weekStart (a Monday) and revenue. Returns 201; 400 for invalid input; 404 for unknown movie; 409 for a duplicate movie and week	Prakash
PUT /performance/{recordId}	Correct a mistaken entry	Body holds revenue only, so the movie and week cannot change. 200 on success; 400; 404 for unknown record	Prakash
DELETE /performance/{recordId}	Remove a record	204 on success; 404 for unknown record	Prakash
Reports			
GET /reports/weeks	List weeks that hold data	200 with a list of weekStart values	Prakash
GET /reports/weekly	Weekly report of movie names and revenue	weekStart required; 400 if missing or malformed; 200 with totalRevenue and a list of movies	Prakash
GET /reports/charts/revenue-trend	Revenue per movie over several weeks	Optional movieId limits the result to one movie; 200 with a list of movies, each with weekly chartPoints	Avishek
Management			
GET /management/review-flags	Movies flagged for review	Accepts weekStart, minRevenue and dropPercent; server defaults apply when omitted; flags revenue below the minimum or a fall of at least dropPercent from the previous week. Each flag carries a reason	Prakash, Avishek
POST /movies/{movieId}/screening/extend	Extend a screening (Screening Extended)	200 with the new screeningStatus; 404 for unknown movie; 409 if the screening has ended	Prakash, Avishek
POST /movies/{movieId}/screening/end	End a screening (Screening Ended)	200 with the new screeningStatus; 404 for unknown movie	Prakash, Avishek
Authentication (optional)			
POST /auth/login	Log in with username and password	Returns token and expiresIn (seconds); 401 on failure; passwords never logged	All pages

3.2 Movie Endpoints and Other Additions to the Contract

The Blueprint shows a movie controller on /movies (Blueprint Figure 5) and refers to POST /movies returning 201 (Blueprint Table 9), yet Blueprint Table 6 does not list the movie operations in any endpoint group, and Blueprint Table 3 has no event for adding a movie even though the event storming narrative says a manager adds one. The input page needs a list of movies to choose from, and the screening endpoints depend on movieId.

The final openapi.yaml therefore adds GET /movies, POST /movies and GET /movies/{movieId}. It also fixes some details that the Blueprint leaves open, such as the names of response fields. Table 5 lists these additions so that the team can confirm them in the pull request and update Blueprint Table 6 if they agree.

Table 5: Additions and choices in the final contract beyond the Blueprint

Addition	Where	Reason
Movie operations: GET /movies, POST /movies, GET /movies/{movieId}	Movies	The input page needs a movie list and the screening endpoints need a movieId
Response fields movies and totalRevenue	GET /reports/weekly	The Blueprint does not name the fields of the weekly report
chartPoints (weekStart, revenue) for each movie, and an optional movieId filter	GET /reports/charts/revenue-trend	Lets the charts show one movie or several
Flag fields movieId, title, revenue and reason	GET /management/review-flags	Tells the manager why each movie was flagged
Screening response with movieId, screeningStatus and message	Extend and end endpoints	Confirms the new status to the page
PUT body holds revenue only	PUT /performance/{recordId}	The movie and week cannot change, so a correction cannot create a duplicate

3.3 Open Decisions and Dependencies

- **Review thresholds.** The rule is agreed but the values of minRevenue and dropPercent are not. I will implement them as request parameters with defaults held in one configuration location, so a later decision requires no controller changes.
- **Authentication.** Login is optional and the bearer-token scheme is defined but not enforced. If login is built, I will remove the empty security option so that every endpoint except login requires a token sent as Authorization: Bearer <token> (Jones & Hardt, 2012). The contract describes the token as a JWT, which I will confirm with the team if login goes ahead.
- **Response shapes.** The field names in Table 5 are my proposals. I will confirm them with Prakash and Avishek, who build the pages that read these responses, and I will record the outcome in the pull request.
- **Roles.** Employee and manager roles are proposed but are outside version 1.0.0 of the contract. Adding them would introduce a 403 Forbidden response and would need a reviewed contract change.

4. Risk Management and Delivery Roadmap

4.1 Risk Register

Table 6 records the principal risks to my module. Likelihood and impact are rated Low, Medium or High on my own judgement of the project at this stage and will be revisited at each phase review.

Table 6: Personal risk register for the API module

ID	Risk	Likelihood	Impact	Mitigation	Contingency
R1	The platform team becomes a bottleneck because every data need passes through Rose and me	Medium	High	Keep the contract stable; collaborate while it is agreed; publish examples so pages can use mock data	Deliver input and report endpoints first; defer charts and optional features
R2	Implementation drifts from openapi.yaml and the pages break	Medium	High	Validate against the contract; check the file in Swagger Editor after every change; test against documented examples	Fix code to match the contract unless a reviewed pull request changes it
R3	Movie endpoints are in the contract but not yet in the Blueprint, so the team may not accept them	Medium	Medium	Explain the need in the pull request and agree the set with Prakash, Avishek and Rose	Keep only the operations on /movies, which the Blueprint already refers to
R4	Review thresholds remain unconfirmed	Medium	Medium	Use configurable server defaults; seek confirmation at the next team meeting	Document the defaults used in the README
R5	Duplicate or invalid performance records	Medium	Medium	Enforce uniqueness in the API and through Rose's database constraint; return 409 or 400	Correct with PUT; use a clean-up script for dummy data
R6	Error messages leak internal details	Low	High	Central error handler returns only code and message; log details server-side	Review responses and logs before submission
R7	Login is added late, and forces changes across endpoints	Medium	Medium	Implement the token check as one optional layer, not in each controller	Exclude login from version 1.0.0 and state it as out of scope
R8	Database schema changes delay integration	Medium	Medium	Agree the data model early; keep all SQL in the data access layer	Use an in-memory stub of the repositories
R9	A breaking contract change disrupts other members	Low	High	Apply the versioning rule: removed field or renamed path raises the major version	Revert the pull request and reissue as a compatible change
R10	No cover for the API during absence	Low	High	Keep the contract and README clear; share endpoint knowledge with Rose at reviews	Ask Rose or Reinaldo to cover merges and basic fixes

4.2 Risk Assessment

Figure 2 plots the register on a likelihood and impact grid. Two risks fall in the highest-priority cells and receive attention first: the bottleneck risk (R1) and contract drift (R2). R3, the movie endpoints, now sits among the medium risks because the operations are already written into the contract and only need the team's agreement.

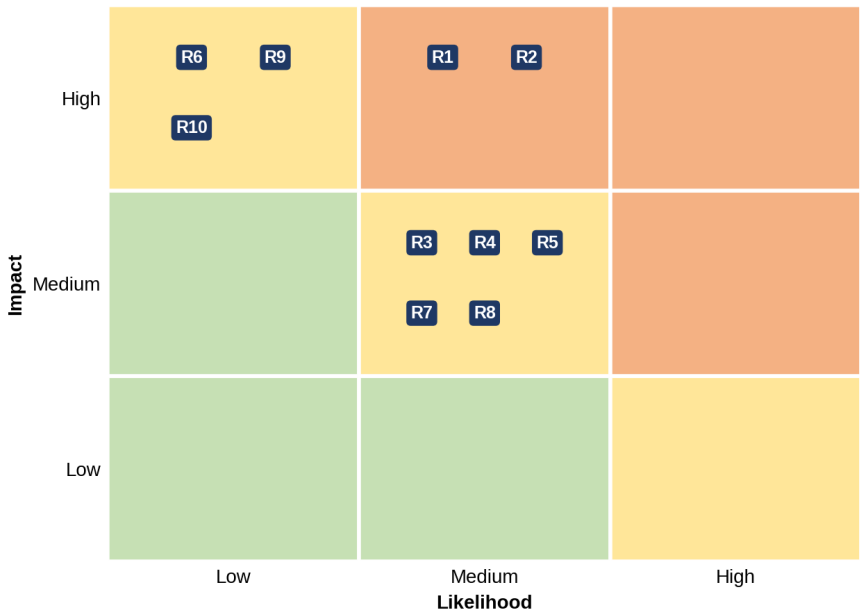


Figure 2: Risk matrix for the API module

4.3 Delivery Roadmap

The Blueprint does not fix dates, so the roadmap is organised in six overlapping phases that I will map onto the assessment timetable (Figure 3). Table 7 gives the deliverables and the check that must be met before a phase is treated as complete.

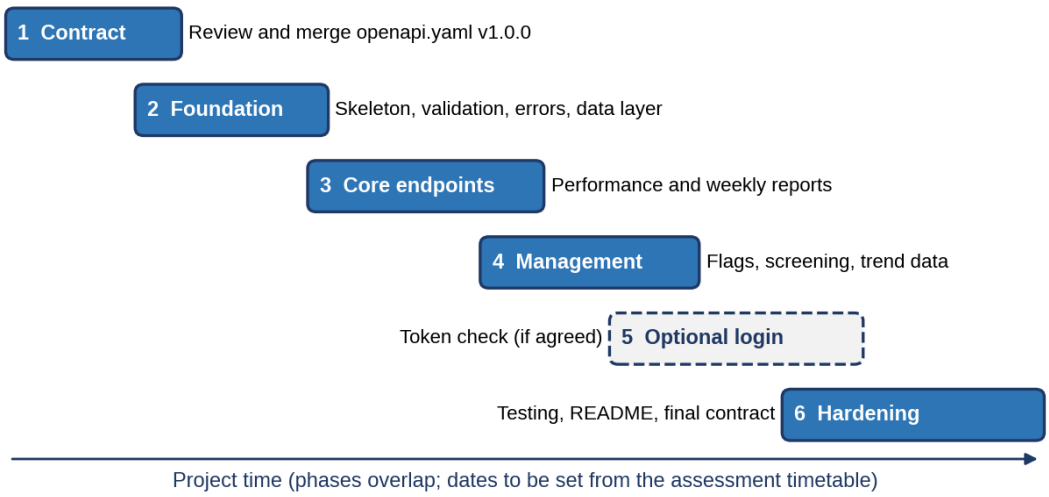


Figure 3: Phased delivery roadmap

Table 7: Delivery phases, deliverables and exit checks

Phase	Focus	Deliverables	Exit check
1. Contract	Review and merge openapi.yaml, including the movie operations and an error response for every endpoint	Version 1.0.0 merged to main (13 operations on 11 paths)	Opens in Swagger Editor with no errors; Prakash and Avishek confirm they can build against it
2. Foundation	Project skeleton, base path /api/v1, validation, error handler, data access layer	Running API returning standard errors	Invalid input returns 400; unknown ids return 404
3. Core endpoints	Performance and weekly report endpoints	POST, PUT, DELETE /performance; GET /reports/weeks and /reports/weekly	Input page saves a record; report page shows a week; a duplicate returns 409
4. Management and charts	Review flags, screening decisions, revenue trend	GET /management/review-flags; extend and end endpoints; GET /reports/charts/revenue-trend	Flags match hand-calculated examples; extending an ended screening returns 409
5. Optional login	Auth service and token check, if agreed	POST /auth/login; token enforcement	Missing or expired token returns 401; passwords never stored in plain text or logged
6. Hardening and submission	Testing, log review and documentation	Test results, README, final contract	No stack traces or queries in any response; Blueprint tables still match the contract

4.4 Verification and Testing Plan

- Test each endpoint against the examples in openapi.yaml, covering one success case and every documented error case.
- Include boundary cases: revenue of zero, revenue with more than two decimal places, a weekStart that is not a Monday, a malformed date and a missing parameter.
- Include state cases: a duplicate movie and week, extending an ended screening and deleting the same record twice.
- Confirm that PUT /performance/{recordId} accepts revenue only, and that GET /movies, POST /movies and GET /movies/{movieId} behave as documented.
- Check that each response uses the field names in the contract, for example totalRevenue, movies and chartPoints.
- Check the review rule with hand-calculated examples for both conditions: revenue below minRevenue, and a fall of dropPercent or more from the previous week.
- Re-open openapi.yaml in Swagger Editor after every contract change to confirm it remains error-free.

5. Governance and Alignment with Team Git Rules

5.1 Position in the Team Structure

I belong to the Data and API Platform Team with Rose. The Blueprint describes our relationship with the stream-aligned Report Experience Team (Prakash and Avishek) as X-as-a-Service: they call the API as defined in openapi.yaml without needing to know how the database is built. While the contract is being agreed, the two teams collaborate closely and change openapi.yaml by pull request. Reinaldo, as the enabling team, facilitates repository rules and usability guidance (Skelton & Pais, 2019).

5.2 Git Working Rules

Table 8 sets out how I will apply the team's repository rules to my own work. Where Reinaldo, as Git administrator, issues more detailed rules such as branch naming, I will follow those.

Table 8: Git working rules applied to the API module

Rule	How I apply it
Contract location	openapi.yaml stays in the main folder of the repository and states the current version
Pull requests	No contract or code change is committed directly to main; every change goes through a pull request
Review and merge	Affected teammates review the change (Prakash and Avishek for response shapes, Rose for data fields) and Reinaldo merges it into main
Validation	The contract must open in Swagger Editor with no errors before a merge is requested
Branching	One short-lived branch per task, named by purpose (for example feature/performance-endpoints), to keep reviews small
Commit messages	Short and specific, naming the endpoint or rule that changed
Versioning	A breaking change (removing a field or renaming a path) raises the major version; a compatible addition raises the minor version
Secrets	No passwords, tokens or credentials are committed to the repository

5.3 Contract Change Process

Figure 4 summarises the process I will follow for any change to the contract. A change begins with a conversation with the affected teammate, is made on a branch, validated, reviewed through a pull request and merged by the Git administrator. I then update the implementation and tests and tell the report team that the change is available.

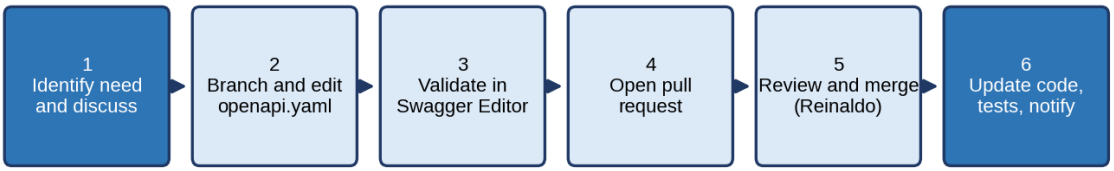


Figure 4: Contract change process

5.4 Communication and Review

- Raise any new data request from the web application as a pull request or issue so that it is visible to Rose and me.
- Hold a short check with the report team after each roadmap phase to confirm that endpoints behave as documented.
- Agree the review thresholds and the login decision with the whole team, as these are the two open matters named in the Blueprint conclusion.
- Treat repository administration as a shared duty that others can cover, as the Blueprint recommends, so my work is not blocked if the Git administrator is unavailable.

6. Conclusion

My module is the REST API that connects the pages to the data. I am responsible for the performance, report, management and optional authentication endpoints in openapi.yaml, and I have added the movie operations to the contract, to be confirmed through a reviewed pull request. The main risks are becoming a bottleneck, drift from the contract, unconfirmed review thresholds and a late decision on login. I will reduce them by keeping the contract stable, enforcing rules in one place, using configurable defaults and following the team's pull-request and versioning rules.

References

Bray, T. (Ed.). (2017). The JavaScript Object Notation (JSON) data interchange format (RFC 8259). Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc8259

Brown, S. (n.d.). The C4 model for visualising software architecture. https://c4model.com

Fielding, R. T., Nottingham, M., & Reschke, J. (Eds.). (2022). HTTP semantics (RFC 9110). Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc9110

Jones, M., & Hardt, D. (2012). The OAuth 2.0 authorization framework: Bearer token usage (RFC 6750). Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc6750

OpenAPI Initiative. (2020). OpenAPI specification (Version 3.0.3). https://spec.openapis.org/oas/v3.0.3

OWASP Foundation. (2023). OWASP API security top 10: 2023. https://owasp.org/API-Security/editions/2023/en/0x00-header/

Skelton, M., & Pais, M. (2019). Team Topologies: Organizing business and technology teams for fast flow. IT Revolution Press.